Assignment 1 - Data Loading, Storage and File Formats

In [1]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

CSV file

In [2]:
```
df_csv = pd.read_csv("sales_csv.csv")

print("CSV Data:")
display(df_csv.head())
```

```
CSV Data:
```

	ORDERNUMBER	QUANTITYORDERED	PRICEEACH	ORDERLINENUMBER	SALES	ORDERDATE	STATUS	QTR_ID	MONTI
0	10107	30	95.70	2	2871.00	2/24/2003 0:00	Shipped	1	
1	10121	34	81.35	5	2765.90	5/7/2003 0:00	Shipped	2	
2	10134	41	94.74	2	3884.34	7/1/2003 0:00	Shipped	3	
3	10145	45	83.26	6	3746.70	8/25/2003 0:00	Shipped	3	
4	10159	49	100.00	14	5205.27	10/10/2003 0:00	Shipped	4	

5 rows × 25 columns

In [4]:
```
print("Shape:", df_csv.shape)
print("Columns:", df_csv.columns.tolist())
```

```
Shape: (2823, 25)
Columns: ['ORDERNUMBER', 'QUANTITYORDERED', 'PRICEEACH', 'ORDERLINENUMBER', 'SALES', 'ORDERDATE', 'STATUS',
'QTR_ID', 'MONTH_ID', 'YEAR_ID', 'PRODUCTLINE', 'MSRP', 'PRODUCTCODE', 'CUSTOMERNAME', 'PHONE', 'ADDRESSLIN
E1', 'ADDRESSLINE2', 'CITY', 'STATE', 'POSTALCODE', 'COUNTRY', 'TERRITORY', 'CONTACTLASTNAME', 'CONTACTFIRS
TNAME', 'DEALSIZE']
```

Excel file

In [3]:
```
df_excel = pd.read_excel("sales_xls.xlsx")

print("Excel Data:")
display(df_excel.head())
```

Excel Data:

	ORDERNUMBER	QUANTITYORDERED	PRICEEACH	ORDERLINENUMBER	SALES	ORDERDATE	STATUS	QTR_ID	MONTI
0	10107	30	95.70	2	2871.00	2/24/2003 0:00	Shipped	1	
1	10121	34	81.35	5	2765.90	5/7/2003 0:00	Shipped	2	
2	10134	41	94.74	2	3884.34	7/1/2003 0:00	Shipped	3	
3	10145	45	83.26	6	3746.70	8/25/2003 0:00	Shipped	3	
4	10159	49	100.00	14	5205.27	10/10/2003 0:00	Shipped	4	

5 rows × 25 columns

In [5]:
```
print("Shape:", df_excel.shape)
print("Columns:", df_excel.columns.tolist())
```

```
Shape: (2823, 25)
Columns: ['ORDERNUMBER', 'QUANTITYORDERED', 'PRICEEACH', 'ORDERLINENUMBER', 'SALES', 'ORDERDATE', 'STATUS',
'QTR_ID', 'MONTH_ID', 'YEAR_ID', 'PRODUCTLINE', 'MSRP', 'PRODUCTCODE', 'CUSTOMERNAME', 'PHONE', 'ADDRESSLIN
E1', 'ADDRESSLINE2', 'CITY', 'STATE', 'POSTALCODE', 'COUNTRY', 'TERRITORY', 'CONTACTLASTNAME', 'CONTACTFIRS
TNAME', 'DEALSIZE']
```

Json file

In [7]:
```
df_json = pd.read_json("sales_json.json")

print("JSON Data:")
display(df_json.head())
```

JSON Data:

	ORDERNUMBER	QUANTITYORDERED	PRICEEACH	ORDERLINENUMBER	SALES	ORDERDATE	STATUS	QTR_ID	MONTI
0	10107	30	95.70	2	2871.00	2/24/2003 0:00	Shipped	1	
1	10121	34	81.35	5	2765.90	5/7/2003 0:00	Shipped	2	
2	10134	41	94.74	2	3884.34	7/1/2003 0:00	Shipped	3	
3	10145	45	83.26	6	3746.70	8/25/2003 0:00	Shipped	3	
4	10159	49	100.00	14	5205.27	10/10/2003 0:00	Shipped	4	

5 rows × 25 columns

In [8]:
```
print("Shape:", df_json.shape)
print("Columns:", df_json.columns.tolist())
```

```
Shape: (2823, 25)
Columns: ['ORDERNUMBER', 'QUANTITYORDERED', 'PRICEEACH', 'ORDERLINENUMBER', 'SALES', 'ORDERDATE', 'STATUS',
'QTR_ID', 'MONTH_ID', 'YEAR_ID', 'PRODUCTLINE', 'MSRP', 'PRODUCTCODE', 'CUSTOMERNAME', 'PHONE', 'ADDRESSLIN
E1', 'ADDRESSLINE2', 'CITY', 'STATE', 'POSTALCODE', 'COUNTRY', 'TERRITORY', 'CONTACTLASTNAME', 'CONTACTFIRS
TNAME', 'DEALSIZE']
```

In [9]:
```
# 1. Combine all three files into one dataframe
df = pd.concat([df_csv, df_excel, df_json], ignore_index=True)

# 2. Remove duplicate rows
df = df.drop_duplicates()

# 3. Fill missing numbers with the average (mean)
df = df.fillna(df.mean(numeric_only=True))
```

```
# 4. Fill missing text columns with the most frequent value (mode)
df = df.fillna(df.mode().iloc[0])

print("Missing values left:", df.isnull().sum().sum())
print("Total rows:", len(df))
```

```
Missing values left: 0
Total rows: 7876
```

Data transformation

In [11]:
```
# Derive a new column: Total Amount
if 'QUANTITYORDERED' in df.columns and 'PRICEEACH' in df.columns:
    df['Total_Amount'] = df['QUANTITYORDERED'] * df['PRICEEACH']

# Split date into year and month
if 'ORDERDATE' in df.columns:
    df['Date'] = pd.to_datetime(df['Date'])
    df['Month'] = df['Date'].dt.month_name()

df.head()
```

Out[11]:

	ORDERNUMBER	QUANTITYORDERED	PRICEEACH	ORDERLINENUMBER	SALES	ORDERDATE	STATUS	QTR_ID	MON
0	10107	30	95.70	2	2871.00	2/24/2003 0:00	Shipped	1	
1	10121	34	81.35	5	2765.90	5/7/2003 0:00	Shipped	2	
2	10134	41	94.74	2	3884.34	7/1/2003 0:00	Shipped	3	
3	10145	45	83.26	6	3746.70	8/25/2003 0:00	Shipped	3	
4	10159	49	100.00	14	5205.27	10/10/2003 0:00	Shipped	4	

5 rows × 28 columns

Analysis & Statistics

In [12]:
```
# Summary statistics for numeric data
print("--- Summary Statistics ---")
print(df.describe())

# Key metrics
sales_col = 'Total_Amount' if 'Total_Amount' in df.columns else df.select_dtypes(include='number').columns[

print("\nTotal Sales:", df[sales_col].sum())
print("Average Order Value:", df[sales_col].mean())

# Group by country
if 'COUNTRY' in df.columns:
    print("\n--- Sales by Country ---")
    print(df.groupby('COUNTRY')[sales_col].sum())
```

```
--- Summary Statistics ---
        ORDERNUMBER  QUANTITYORDERED    PRICEEACH  ORDERLINENUMBER  \
count   7876.000000      7876.000000  7876.000000      7876.000000
mean   10258.502666        35.153885    83.630973         6.453403
min    10100.000000         6.000000    26.880000         1.000000
25%    10178.000000        27.000000    68.710000         3.000000
50%    10262.000000        35.000000    95.800000         6.000000
75%    10333.000000        43.000000   100.000000         9.000000
max    10425.000000        97.000000   100.000000        18.000000
std       92.267563         9.765696    20.248462         4.225154

              SALES       QTR_ID     MONTH_ID      YEAR_ID         MSRP  \
count   7876.000000  7876.000000  7876.000000  7876.000000  7876.000000
mean    3562.555086     2.714195     7.081767  2003.815515   100.772473
min      482.130000     1.000000     1.000000  2003.000000    33.000000
25%     2204.600000     2.000000     4.000000  2003.000000    68.000000
50%     3185.710000     3.000000     8.000000  2004.000000    99.000000
75%     4514.920000     4.000000    11.000000  2004.000000   124.000000
max    14082.800000     4.000000    12.000000  2005.000000   214.000000
std     1850.215098     1.203680     3.657518     0.701105    40.303667

       Total_Amount                          Date
count   7876.000000                          7876
mean    2941.018441  2004-05-10 20:39:36.841036160
min      482.130000           2003-01-06 00:00:00
25%     2100.000000           2003-11-07 00:00:00
50%     2800.000000           2004-06-15 00:00:00
75%     3756.000000           2004-11-17 00:00:00
max     9048.160000           2005-05-31 00:00:00
std     1109.493612                           NaN

Total Sales: 23163461.240000002
Average Order Value: 2941.0184408329105

--- Sales by Country ---
COUNTRY
Australia      1145735.72
Austria         518379.15
Belgium         189057.76
Canada          387008.68
Denmark         578242.89
Finland         717390.84
France         2757773.55
Germany         536067.24
Ireland         129711.72
Italy           928208.61
Japan           306153.38
Norway          587455.50
Philippines     160582.34
Singapore       683956.50
Spain          3065117.91
Sweden          406902.41
Switzerland     280034.73
UK              826406.68
USA            8959275.63
Name: Total_Amount, dtype: float64
```

Visualization

In [16]:

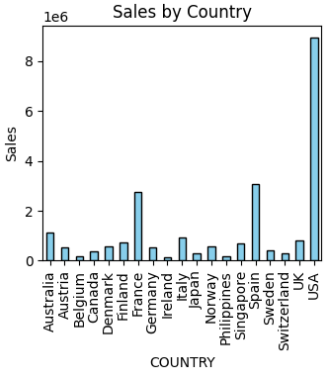
```python
plt.figure(figsize=(12, 4))

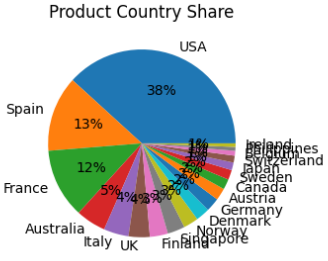
# 1. Bar Chart: Sales by Country
plt.subplot(1, 3, 1)
if 'COUNTRY' in df.columns:
    df.groupby('COUNTRY')[sales_col].sum().plot(kind='bar', color='skyblue', edgecolor='black')
    plt.title('Sales by Country')
    plt.ylabel('Sales')

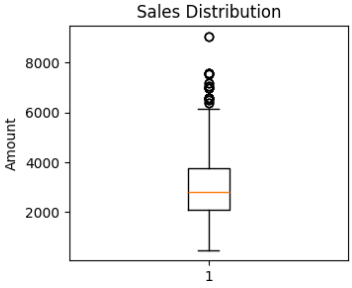
# 2. Pie Chart: Country Distribution
plt.subplot(1, 3, 2)
if 'COUNTRY' in df.columns:
    df['COUNTRY'].value_counts().plot(kind='pie', autopct='%1.0f%%')
    plt.title('Product Country Share')
    plt.ylabel('')
```

```
# 3. Box Plot: Sales Spread
plt.subplot(1, 3, 3)
plt.boxplot(df[sales_col].dropna())
plt.title('Sales Distribution')
plt.ylabel('Amount')

plt.tight_layout()
plt.show()
```







In [ ]: